

Three Ways Input-to-State Mutation *Hurts*: A Byte-Level Account of the I2S-Anti Mechanisms

BlockerAnalyzer benchmark — I2S-anti family

2026-06-13

Abstract

In the BlockerAnalyzer benchmark, 41 decisive blocker branches are labeled *I2S-anti*: branches the input-to-state (I2S / cmplog) fuzzer *fails* to reach while a non-I2S arm (naive havoc or value-profile) succeeds. All three I2S-anti sub-mechanisms share one measured signature — on the real campaign corpus, the cmplog arm is *depleted* of the byte value the winning arm reaches — but they differ in *why*. This note defines the measurement at the byte level, then walks one specific branch per mechanism with its real gate offset(s), byte values, and corpus statistics.

1 The shared measurement: operand_enrichment

Every I2S-anti label is decided by one tool, `i2s_operand_availability.py` (registry name `operand_enrichment`), run directly on the two campaign corpora. The tool reads no source and assumes no mechanism; it measures bytes:

1. **Gate signature.** Compare the branch’s *resolving* (W) seeds against its *blocking* (L) seeds. The *gate offsets* are the byte positions at which W and L seeds systematically disagree (consensus purity $\geq$ threshold). At each gate offset, the W-consensus byte is the **target** pattern, the L-consensus byte is the **decoy** pattern.
2. **Corpus fractions.** For each fuzzer $F \in \{\text{cmplog}, \text{naive}\}$, scan F ’s saved corpus and compute `target_fracF` (fraction whose head matches the full target pattern at all gate offsets) and `decoy_fracF` (fraction matching the decoy pattern).
3. **Signed enrichment.**

$$\text{signed_target_enrich} = \log_2 \frac{\text{target_frac}_{\text{cmplog}}}{\text{target_frac}_{\text{naive}}}, \quad \text{decoy_enrich} = \log_2 \frac{\text{decoy_frac}_{\text{cmplog}}}{\text{decoy_frac}_{\text{naive}}}.$$

A value `signed_target_enrich` < 0 means cmplog’s corpus contains the target byte *less* often than naive’s — I2S is steering bytes *away* from the value that resolves the branch. All three labels require `signed_target_enrich` < -0.3 ; they are separated by the decisive *shape* (which fuzzers win/lose) and by *what* the decoy is.

2 Mechanism 1 — target_depletion: a rival operand wins the harvest

Example branch: `libpng_3977` (validated, $n = 23$ for this label).

Table 1: The three I2S-anti mechanisms at a glance. Shape code is over [cmplog, vp, vpc, naive] with W=win / L=loss / _=non-decisive.

label	shape	decoy is...	rule
i2s_decoy_substitution_target_depletion	LW_W	a competing real operand	signed_target_enrich
i2s_anti_structural_byte_corruption	LWWW	— (blind overwrite)	signed_target_enrich
i2s_decoy_substitution_overfit	LWLW	a structural “stay-valid” literal	signed_target_enrich

The gate is a single byte. `operand_enrichment` found `gate_offsets = [236]`, `gate_width = 1`. The branch is the `CHRM` (chromaticity) chunk handler comparing a parsed fixed-point value to the error sentinel:

Listing 1: `/src/libpng/pngutil.c:1283`, `OSS_FUZZ_png_handle_CHRM`. The gate byte at input offset 236 feeds `xy.blueX`.

```

1  if (xy.redx == PNG_FIXED_ERROR ||
2      xy.redy == PNG_FIXED_ERROR ||
3      xy.greenx == PNG_FIXED_ERROR ||
4      xy.greeny == PNG_FIXED_ERROR ||
5      xy.blueX == PNG_FIXED_ERROR || /* <-- decisive branch 3977 */
6      xy.bluey == PNG_FIXED_ERROR)
7      /* error path: chunk rejected */

```

At offset 236 the resolving seeds carry the **target byte** `0x00`; the blocking seeds carry the **decoy byte** `0x65` (ASCII ‘e’). The decoy is not random: it is the exponent character of ASCII numeric literals (“1.0e0”, “65.535”) that `cmplog` harvests from *other* string/number comparisons in the PNG parser and splices in via `I2SRandReplace`:

Listing 2: Hex window over bytes 228–244 around the gate byte (in brackets at offset 236). Top: a resolving (W) seed. Bottom: a blocking (L) `cmplog` seed — note the embedded ASCII float 1.0e0 whose e (0x65) lands exactly on the gate.

```

W (resolving, naive) : 02 02 02 02 02 75 30 00 [00] ea 60 20 00 79 98 00
00
L (blocking, cmplog) : 2f 62 61 72 00 31 2e 30 [65] 30 00 36 35 2e 35 33
35
                        ( "1.0e0" )                ( "65.535" )

```

Table 2: `libpng_3977` operand.enrichment metrics (`--sample 8000 --head 256`). `cmplog` is depleted of the target `0x00` and enriched in the decoy `0x65`.

quantity	cmplog	naive
corpus size scanned	4561	4447
target_frac (byte 0x00 @236)	0.0886	0.1565
decoy_frac (byte 0x65 @236)	0.6290	0.3929
signed_target_enrich = $\log_2(0.0886/0.1565) = -0.821$ (target depleted)		
decoy_enrich = $\log_2(0.6290/0.3929) = +0.679$ (decoy enriched)		

Reading: a real, abundant competing operand (the textual `0x65`) exists at the same byte position. `cmplog` logs it and `RandReplaces` it in, *pinning* 63% of its corpus to the decoy and dropping the

target 0x00 to 8.9% (vs. 15.7% for naive, which never harvests the decoy and drifts onto the target freely). The extra rule clause `skip != 'no_gate_signature'` guarantees a genuine two-operand competition, not noise.

3 Mechanism 2 — byte_corruption: blind RandReplace clobbers an assembled byte

Example branch: `harfbuzz_5831` (validated; server-a target).

Shape LWWW: cmplog is the *sole* loser (naive, value_profile, and vpc all win) — the clean fingerprint of “only the I2S mutator breaks this.” Here there is no clever rival operand. Naive havoc *assembles* the correct structural byte; cmplog’s `I2SRandReplace` blindly overwrites that exact position with a logged value, *destroying* the byte rather than redirecting it to a single dominant decoy.

Table 3: `harfbuzz_5831` operand_enrichment metrics. The target byte is depleted in cmplog (`signed_target_enrich < 0`); decoy fractions are small in absolute terms (diffuse corruption), not one dominant rival.

quantity	cmplog	naive
<code>target_frac</code>	0.2113	0.3799
<code>decoy_frac</code>	0.0293	0.0030
<code>signed_target_enrich</code> = −0.882 (target byte clobbered out of cmplog’s corpus)		
<code>decoy_enrich</code> = +2.64 (weak, low-absolute — diffuse, not a single decoy)		

Reading: the structural byte is present in 38% of naive’s corpus but only 21% of cmplog’s — I2S keeps overwriting it. Contrast with Mechanism 1: there the decoy is a *single abundant* value (`decoy_frac`= 0.63); here the decoy mass is small and scattered (`decoy_frac`= 0.029), the signature of *corruption* (random clobber) rather than *competition* (capture by one rival).

Byte-level proof (server a). Mechanism 2 is *defined* by the absence of a single decoy, so — unlike M1 and M3 — there is no clean “target → one decoy” byte to exhibit; the evidence *is* the diffuseness. Recomputed on harfbuzz’s corpus, the gate is wide and scattered (`gate_width`= 29, offsets 71–187) — itself a tell that no single byte is decisive. The assembled structural byte is depleted in cmplog and present in naive (per-offset median `target_frac` 0.20 vs. 0.34, reproducing the assignment’s 0.21/0.38), but the bytes cmplog writes there do *not* concentrate on any replacement:

Listing 3: Byte distribution at gate offset 71 (assembled target 0x00) over the 8000-input corpora. naive peaks on the target; cmplog is spread over 211 distinct values with no dominant substitute — the fingerprint of blind corruption, not a decoy.

naive	@71	:	0x00=0.39	0xff=0.03	0x05=0.03	0x01=0.03	...	(236 distinct)
cmplog	@71	:	0x00=0.32	0x10=0.12	0x01=0.07	0x04=0.06	...	(211 distinct)

Across the 29 gate offsets the *largest single non-target byte* in cmplog reaches only ≈ 0.12 (median), versus the one dominant decoy of M1 (0.63) and M3 (0.51): nothing to capture, nothing re-asserted, only scatter. Seed length corroborates the assembly story — cmplog’s blocking seeds are short (10–61 B, one 405 B outlier) while the resolving naive seeds run 60–188 B: cmplog *truncates* / *fails to*

assemble the record rather than substituting one wrong value into a complete one. This is why M2, alone among the three, has no clean two-row decoy window: corruption is diffuse by construction.

Caveat: why naive resolves despite its own noise, and what that implies for the label.

A fair objection: naive’s corpus is *also* noisy at the gate — at offset 71 it has *more* distinct bytes (236 vs. 211) and only a slightly higher target share (0.39 vs. 0.32). So “naive is cleaner” is *not* why it wins. Resolution does not require a low-noise corpus; it requires *one* input that assembles the structure and is *kept* by coverage feedback. The discriminator here is structural completeness, visible in seed length: naive’s resolving seeds run 60–188 B while cmplog’s blocking seeds are 10–61 B (one 405 B outlier) — and *all* 29 gate offsets lie at byte ≥ 71 , so eight of cmplog’s nine blocking seeds are shorter than the *first* gate offset. Naive’s plain havoc occasionally completes the record and coverage latches the long input; cmplog’s I2SRandReplace overwrites bytes mid-assembly, so its corpus stays short and broken and never latches a resolving input. The per-byte scatter is the residue of that repeated clobber, not the cause.

This refines the label rather than contradicting it. The winner/loser median seed-length ratio is ≈ 1.7 for `harfbuzz.5831` (cmplog truncated) — the *opposite* sign from the M3 overfit case below, where cmplog’s blocking seeds are *longer* (ratio ≈ 0.4 , bloated “stay-valid” inputs). `operand_enrichment` cannot see this: it reports `signed_target_enrich` < 0 for both. So `harfbuzz.5831` sits between pure in-place byte-corruption and an *assembly/truncation failure* — the byte is deleted because the record never completes, not (only) because a complete record is clobbered. A `size_lift` (winner-vs-loser seed size) clause would subtype these honestly, but the present I2S-anti member set is too small ($n=8$ for this label) to fit such a split without over-anchoring on a single branch, so it is left as a noted refinement rather than a new mechanism.

4 Mechanism 3 — overfit: capture by a “stay-valid” literal, never exploring the free byte

Example branch: `harfbuzz.5323` (validated; server-a target).

Shape LWLW: both I2S carriers (cmplog and, by the shape, the value-profile arm) lose; the winning gate is a *free byte* that naive/VP can hit by random mutation. cmplog instead gets *captured* by a *structural literal* elsewhere in the input — it keeps splicing that literal to keep the input well-formed, and as a result *never randomizes* the free-byte target. It “overfits” to structural validity.

Table 4: `harfbuzz.5323` `operand_enrichment` metrics — the extreme case. cmplog’s corpus is half the structural decoy; naive’s has almost none.

quantity	cmplog	naive
<code>target_frac</code>	0.0816	0.5083
<code>decoy_frac</code>	0.5075	0.0004
<code>signed_target_enrich</code> = −2.64 (target free-byte present in 51% of naive vs 8% of cmplog)		
<code>decoy_enrich</code> = +10.4 (structural literal dominates cmplog, near-absent in naive)		

Reading: this is the strongest anti-signal in the family. cmplog spends 51% of its corpus re-asserting the structural literal (`decoy_frac`_{cmplog} = 0.5075 vs. 0.0004 for naive) and collapses the winning free byte to 8% (vs. 51% for naive). Where Mechanism 1 is *two operands competing* and Mechanism 2 is *one byte being corrupted*, Mechanism 3 is a *validity trap*: I2S is “succeeding” at keeping the input structurally valid and thereby starving the one position that actually matters.

Byte-level proof (server a). Recomputing the gate directly on harfbuzz’s corpus identifies the structural decoy concretely: it is the **first OpenType table tag** — the 4-byte field at input offsets [12:16], immediately after the 12-byte `sfnt` header (`sfntVersion(4)+numTables(2)+searchRange(2)+entrySel`). In cmplog’s blocking seeds those four bytes spell a *registered* tag — GPOS, GSUB, or MATH; in the resolving (naive) seeds the same position is unstructured:

Listing 4: Hex window over bytes 8–23. The 4-byte table tag (brackets, offsets 12–15) is the literal GPOS (47 50 4f 53) in the cmplog seed and random in the naive seed. The following 8 bytes are the table record’s `checksum` and `offset`.

W (resolving, naive)	: ea 00 00 ff	[00 1b 00 00]	63 61 72 66 15 03 00 0e
L (blocking, cmplog)	: 21 20 1e 20	[47 50 4f 53]	00 01 00 0d 00 00 00 10
		("GPOS")	(checksum) (tbl off)

At corpus scale the tag is overwhelmingly an I2S artifact. Of cmplog’s 8000-input sample, **50.3%** carry exactly GPOS at offsets 12–15 and **82.9%** carry *some* printable table tag there; for naive the figures are **0.0%** and **6.1%**. This reproduces the tool’s `decoy_frac` (0.508 vs. 0.0004) at the byte level. Crucially, the “literal” is a *tag field*, not one fixed constant: cmplog harvests whichever registered tag the OpenType CMP checks expose (GPOS/GSUB/MATH) and re-asserts it to keep the font’s table directory well-formed — precisely why the signature is “structural” rather than a single magic number.

A caveat on reading the gate tool here. Run at the default consensus purity (0.6), `operand_enrichment` reports a 3-offset, *non-contiguous* gate ([11,14,15], decoy bytes " OS") — a *fragment* of the tag, not the whole field. This is structural to the measurement, not a contradiction: the gate keeps only offsets where *both* W and L seeds reach consensus, but the resolving side here is heterogeneous naive input with *no* consensus at offsets 12–13, so those positions drop out and only the O,S tail of GPOS survives. The full 4-byte tag is recovered by dumping the cmplog (L) seeds directly (above) or by the corpus-fraction test — both shown here.

5 Side-by-side

Table 5: The discriminating contrast. All three have `signed_target_enrich` < 0 (target depleted in cmplog); the *decoy* statistics separate them.

	M1 depletion libpng.3977	M2 corruption harfbuzz.5831	M3 overfit harfbuzz.5323
shape	LW_W	LWWW	LWLW
<code>signed_target_enrich</code>	−0.82	−0.88	−2.64
<code>decoy_enrich</code>	+0.68	+2.64	+10.4
<code>decoy_frac</code> cmplog	0.629	0.029	0.508
<code>decoy_frac</code> naive	0.393	0.003	0.0004
decoy character	one abundant rival	diffuse / none	one structural literal
winner gate	competing operand	assembled byte	free byte
one-line	rival wins harvest	byte clobbered	validity trap

Provenance note. `libpng_3977` is a server-sB target; its gate offset, target/decoy bytes, and seed windows above were extracted directly from the on-disk campaign corpus. `harfbuzz_5831` and `harfbuzz_5323` are server-a targets; their metric values are taken verbatim from `step5b_new_v3/*/assignments_s4` and their byte-level evidence (Listings 3 and 4) was extracted directly from server a’s on-disk campaign corpus (`cmplog/naive` queues, `--sample 8000`). The slight differences between the reproduced per-offset fractions and the assignment’s stored values are sampling noise from the random corpus draw.